Reflected XSS with Some SVG Markup Allowed

1. Overview

- **Vulnerability:** Reflected Cross-Site Scripting (XSS)
- **Category:** Web Application Security
- **Severity:** Medium to High (context-dependent)
- **Platform:** PortSwigger Web Security Academy
- **Attack Vector:** Malicious SVG markup with an event handler
- **Status:** Educational Lab Exercise

2. Description

Reflected Cross-Site Scripting occurs when an application takes user-controlled input and returns it in an HTTP response without adequately preventing script execution.

In this lab, the application filters HTML and SVG markup but allows certain SVG elements or attributes to pass through. An attacker may abuse the remaining permitted markup to introduce an event handler that executes JavaScript when the relevant browser event occurs.

SVG (Scalable Vector Graphics) is an XML-based format supported by modern browsers. When SVG markup is embedded in an HTML document, certain elements and event-handler attributes can introduce XSS risks if they are not properly controlled.

3. Vulnerability Details

The application's input filtering is incomplete. Although some potentially dangerous markup is blocked, the remaining SVG elements and attributes may still provide a way to execute JavaScript.

The vulnerability demonstrates that allowing selected markup without a carefully designed sanitization policy can leave an application exposed to XSS.

The security issue is not SVG itself; it is the unsafe handling of user-controlled markup in an executable browser context.

4. Lab Reproduction Steps

1. Launch the assigned PortSwigger Web Security Academy lab.
2. Identify the search functionality that accepts user-controlled input.
3. Submit a normal search term and inspect the resulting page.
4. Determine where the input is reflected in the HTML response.
5. Test the application's filtering behavior using harmless HTML and SVG test cases.
6. Identify which SVG elements and attributes remain permitted.
7. In the authorized lab, construct a test case using an allowed SVG element and a suitable event handler.
8. Submit the test case and verify whether JavaScript executes.
9. Confirm the successful result using the lab's completion indicator.

Expected result: JavaScript execution is possible through permitted SVG markup despite the application's filtering restrictions.

5. Security Impact

Depending on the affected application's functionality and security controls, successful exploitation could allow an attacker to:

- Execute JavaScript in a victim's browser under the vulnerable website's origin.
- Access sensitive page data available to JavaScript.
- Manipulate page content and user interactions.
- Perform actions through an authenticated browser session where applicable protections do not prevent them.
- Create deceptive content that appears to originate from a trusted website.

Actual impact depends on the application context, user privileges, browser behavior, and deployed security controls.

6. Root Cause

The root cause is inadequate sanitization of user-controlled HTML and SVG markup.

The filtering mechanism permits a subset of SVG content without sufficiently restricting dangerous event-handler attributes or ensuring that the resulting markup is safe in its rendering context.

A blacklist-based approach may block known attack patterns while leaving alternative browser-supported execution paths available.

7. Remediation

- Avoid rendering untrusted input as executable HTML or SVG.

- Use context-aware output encoding when displaying user-supplied text.
- Prefer `textContent` over `innerHTML` when HTML rendering is unnecessary.
- If SVG uploads or inline SVG content are required, use a reputable sanitizer configured with a strict allowlist.
- Remove dangerous event-handler attributes and disallow unnecessary SVG elements.
- Validate input on the server side, while recognizing that input validation alone does not replace output encoding.
- Deploy an appropriate Content Security Policy as defense in depth.
- Include SVG elements, event handlers, and filter-bypass cases in security regression testing.

8. References

- PortSwigger Web Security Academy: https://portswigger.net/web-security/cross-site-scripting
- OWASP Cross-Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP Cross-Site Scripting: https://owasp.org/www-community/attacks/xss/

9. Disclaimer

This report is intended for educational and authorized security testing. Exploitation should be limited to designated labs or systems for which permission has been granted.

Reflected XSS with Some SVG Markup Allowed

1. Overview

- **Vulnerability:** Reflected Cross-Site Scripting (XSS)
- **Category:** Web Application Security
- **Severity:** Medium to High (context-dependent)
- **Platform:** PortSwigger Web Security Academy
- **Attack Vector:** Malicious SVG markup with an event handler
- **Status:** Educational Lab Exercise

2. Description

Reflected Cross-Site Scripting occurs when an application takes user-controlled input and returns it in an HTTP response without adequately preventing script execution.

In this lab, the application filters HTML and SVG markup but allows certain SVG elements or attributes to pass through. An attacker may abuse the remaining permitted markup to introduce an event handler that executes JavaScript when the relevant browser event occurs.

SVG (Scalable Vector Graphics) is an XML-based format supported by modern browsers. When SVG markup is embedded in an HTML document, certain elements and event-handler attributes can introduce XSS risks if they are not properly controlled.

3. Vulnerability Details

The application's input filtering is incomplete. Although some potentially dangerous markup is blocked, the remaining SVG elements and attributes may still provide a way to execute JavaScript.

The vulnerability demonstrates that allowing selected markup without a carefully designed sanitization policy can leave an application exposed to XSS.

The security issue is not SVG itself; it is the unsafe handling of user-controlled markup in an executable browser context.

4. Lab Reproduction Steps

1. Launch the assigned PortSwigger Web Security Academy lab.
2. Identify the search functionality that accepts user-controlled input.
3. Submit a normal search term and inspect the resulting page.
4. Determine where the input is reflected in the HTML response.
5. Test the application's filtering behavior using harmless HTML and SVG test cases.
6. Identify which SVG elements and attributes remain permitted.
7. In the authorized lab, construct a test case using an allowed SVG element and a suitable event handler.
8. Submit the test case and verify whether JavaScript executes.
9. Confirm the successful result using the lab's completion indicator.

Expected result: JavaScript execution is possible through permitted SVG markup despite the application's filtering restrictions.

5. Security Impact

Depending on the affected application's functionality and security controls, successful exploitation could allow an attacker to:

- Execute JavaScript in a victim's browser under the vulnerable website's origin.
- Access sensitive page data available to JavaScript.
- Manipulate page content and user interactions.
- Perform actions through an authenticated browser session where applicable protections do not prevent them.
- Create deceptive content that appears to originate from a trusted website.

Actual impact depends on the application context, user privileges, browser behavior, and deployed security controls.

6. Root Cause

The root cause is inadequate sanitization of user-controlled HTML and SVG markup.

The filtering mechanism permits a subset of SVG content without sufficiently restricting dangerous event-handler attributes or ensuring that the resulting markup is safe in its rendering context.

A blacklist-based approach may block known attack patterns while leaving alternative browser-supported execution paths available.

7. Remediation

- Avoid rendering untrusted input as executable HTML or SVG.
- Use context-aware output encoding when displaying user-supplied text.
- Prefer `textContent` over `innerHTML` when HTML rendering is unnecessary.
- If SVG uploads or inline SVG content are required, use a reputable sanitizer configured with a strict allowlist.
- Remove dangerous event-handler attributes and disallow unnecessary SVG elements.
- Validate input on the server side, while recognizing that input validation alone does not replace output encoding.
- Deploy an appropriate Content Security Policy as defense in depth.
- Include SVG elements, event handlers, and filter-bypass cases in security regression testing.

8. References

- PortSwigger Web Security Academy: https://portswigger.net/web-security/cross-site-scripting
- OWASP Cross-Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP Cross-Site Scripting: https://owasp.org/www-community/attacks/xss/

9. Disclaimer

This report is intended for educational and authorized security testing. Exploitation should be limited to designated labs or systems for which permission has been granted.